

Programmation avec PostgreSQL

Les fonctions et procédures

Définition d'une fonction

Une fonction est définie à l'aide de l'instruction CREATE FUNCTION et la structure est la suivante :

```
CREATE (OR REPLACE) FUNCTION nom_fonction(arguments) RETURNS type AS $$  
    DECLARE  
        var1 type;  
    BEGIN  
        /*code de la fonction */  
        RETURN valeur_a_retourner;  
    END;  
  
$$ LANGUAGE plpgsql;
```

La fonction va alors retourner une ou plusieurs valeurs en fonction du type de données prévu.

Différence entre une fonction et une procédure : une procédure ne retourne rien comparé à une fonction.

Appel d'une fonction

L'utilisation d'une fonction (ou procédure) se fait lors de l'écriture d'une requête SQL

```
SELECT nom_fonction(arguments);
```

Définition d'une procédure

Une procédure est définie à l'aide de l'instruction CREATE PROCEDURE et la structure est la suivante :

```
CREATE (OR REPLACE) PROCEDURE nom_procedure(arguments) AS $$  
    DECLARE  
        var1 type;  
    BEGIN  
        /*code de la procédure*/  
    END;  
  
$$ LANGUAGE plpgsql;
```

Appel d'une procédure

```
CALL nom_procedure(arguments);
```

SELECT renvoie quelque chose / CALL ne renvoie rien

Passage de valeurs aux fonctions et procédures

```
SELECT nom_fonction(valeur1, valeur2);
```

Cela nécessite que lors de la déclaration, cela ait été prévu :

```
CREATE FUNCTION nom_fonction(nom_var1 type, nom_var2 type) RETURNS type as $$
```

Surcharge d'une fonction ou procédure

```
CREATE FUNCTION recherche(id_personne integer)  
CREATE FUNCTION recherche-nom_personne char, prenom char)
```

Suppression d'une fonction ou procédure

```
DROP FUNCTION nom_fonction(arguments);
```

Commentaire

On peut utiliser des commentaires sur une seule ligne : – Un commentaire sur une ligne

Variables et structures de données

Affectation des variables

On affecte une valeur à une variable de trois manière :

A l'initialisation : `nom_variable type :=valeur;`

Affectation suite à une opération :

`c := a + b;`

Affectation provenant du résultat d'une requête. On utilise alors la clause INTO :

`SELECT id FROM table WHERE nom ="Einstein" INTO var;`

Ici, la variable var (déclarée auparavant) va stocker le résultat de la requête, càd l'ID correspondant au nom.

Il est possible d'affecter ainsi plusieurs valeurs à la foi :

`SELECT nom_personne, prenom_personne FROM user WHERE id = 4 INTO nom, prenom;`

Accès au contenu des variables

L'accès s'effectue simplement en indiquant le nom de la variable (pas de \$ ou de % devant).

Exemple :

`RETURN nom_variable;`

Constantes

Une constante est une variante d'une variable, qui se différencie par la clause CONSTANT :

`une_constant CONSTANT integer := 4;`

Concaténation

L'opérateur de concaténation est ||

Exemple :

```
res := || 'résultat';
```

Transtypage (cast)

CAST (valeur AS type) – le résultat est la transformation de la valeur dans le type indiqué.

On peut aussi utiliser la syntaxe PostgreSQL non standard suivante :

valeur :: type –même résultat que précédemment

Structures de contrôle

Structure conditionnelle SI ALORS FINSI

```
IF condition THEN
    instruction;
END IF;
```

Structure conditionnelle alternative SI ALORS SINON

```
IF condition THEN
    instruction;
ELSE
    instruction;
END IF;
```

Structure conditionnelle alternative SI ALORS SINON SI

```
IF condition THEN
    instruction;
ELSIF condition THEN
    instruction;
ELSE
    instruction;
END IF;
```

Expression booléennes : <, >, <=, >=, =, <>

Exemple

```
SELECT id from clients where nom='nom' INTO id_var;  
IF id_var>10 THEN  
    RETURN id_var;  
END IF;
```

Les opérateurs booléens sont : AND et OR

On peut aussi tester si une variable possède une valeur grâce à IS NULL (ou IS NOT NULL)

Exemple:

```
SELECT id from clients where nom='nom' INTO id_var;  
IF id_var IS NULL THEN  
    RETURN id_var;  
END IF;
```

Boucle tant que :

```
WHILE condition LOOP  
    instruction;  
END LOOP;
```

Boucle POUR:

```
FOR var IN [REVERSE] val1..val2 LOOP  
    instruction;  
END LOOP;
```

La variable de loop var n'a pas besoin d'être déclaré dans le DECLARE